

NORTHSTAR SPRINT

Project Report

Self-Serve Support Deflection MVP for Northstar Retail Co.

Group 62

Prepared for: Northstar Retail Co. — Support Deflection Assessment

Live prototype: northstar-support-buddy.lovable.app

Repository: github.com/Livingpr/The-Northstar-Sprint-Assignment

Date: 16 August 2026

1. Executive Summary

Northstar Retail Co. asked for a working prototype that lets customers self-resolve common support questions without a human agent in the loop. The Team Charter set the bar at deflecting at least two of three ticket categories: order status, returns & refunds, and stock availability.

The delivered prototype is a Streamlit-based self-serve dashboard (app.py), backed by two CSV data sources (orders.csv and stock.csv). It meets the charter's bar and goes one further: all three target ticket types are implemented and working, plus a fourth self-serve surface — an FAQ page — that catches the remaining long tail of repeat questions.

The app is published as a public GitHub repository with a README covering installation and usage, and is also live at a hosted URL for stakeholder demo access without a local install.

Requirement	Target	Delivered
Ticket categories deflected	At least 2 of 3	3 of 3, plus FAQ page
End-to-end demoable MVP	Yes	Yes — hosted + local install
Source control & docs	Shared repo, README	Public GitHub repo, README with setup/usage
Task sizing (anti-black-box rule)	Every task ≤ 4 hours	10-task board, capped per charter

2. Problem Statement

Northstar's support queue is dominated by repetitive, low-complexity questions: “where is my order,” “when do I get my refund,” and “is this back in stock.” Each of these currently requires a human agent to look up a record that the customer could retrieve themselves, given the right interface.

This pattern drives up response time for genuinely complex tickets and keeps cost-per-contact high for issues that don't require human judgment. The project's goal was to build a tool that answers the majority of these questions directly from existing order and inventory data — deflecting tickets before they are opened, and acting as a first-line replacement for the queue on this subset of intents.

3. Approach to Resolving the Problem

3.1 Ticket Categories Covered

- **Order Status** — customer enters an Order ID and receives item, quantity, order date, current status, and a plain-language next step (e.g. “your order is on its way”).
- **Returns & Refunds** — customer enters an Order ID and receives return status, refund amount, return reason, and a status-specific next step (requested / in transit / received / completed).
- **Stock Availability** — customer searches by product ID or name and receives live quantity, a low-stock warning below the reorder threshold, or a restock date if the item is out of stock.

- **FAQ** — eight pre-written answers to the most common surrounding questions (tracking, non-delivery, return timelines, order changes), so a query that doesn't fit the three structured lookups still gets deflected.

3.2 Design Principle

Every screen follows the same three-step shape: ask for one identifier (Order ID, or product name/ID) → look it up against the data → show status plus a concrete next action. That consistency is what keeps a five-page app usable without an onboarding flow — a customer who learns the Order Status page already understands how Returns & Refunds works.

3.3 Technology Stack

Layer	Choice & Rationale
Frontend / app framework	Python (Streamlit) — chosen for familiarity and speed: a working, styled multi-page UI in one file, with no separate frontend build.
Data layer	Pandas over two flat CSV files (orders.csv, stock.csv) — no database needed at MVP scope; <code>@st.cache_data</code> avoids re-reading on every interaction.
Hosting	Prototype hosted at northstar-support-buddy.lovable.app for stakeholder access without a local install.
Source control	Public GitHub repository (MIT-licensed code, proprietary data files) with a README covering setup, usage, and file structure.

3.4 Data Model

orders.csv holds one row per order: order_id, item, quantity, order_date, order_status, return_status, refund_amount, return_reason. stock.csv holds one row per product: product_id, product_name, quantity_in_stock, reorder_threshold, restock_date. Both files are read once per session and cached, keeping every lookup a simple in-memory filter rather than a live query.

File	Grain	Key Fields
orders.csv	One row per order	order_id, item, quantity, order_date, order_status, return_status, refund_amount, return_reason
stock.csv	One row per product	product_id, product_name, quantity_in_stock, reorder_threshold, restock_date

4. Results

4.1 What Works End-to-End

- **Order Status lookup** — returns item, quantity, order date, and status for a valid Order ID, with color-coded status (delivered / shipped / processing) and a matching next-step message.
- **Returns & Refunds lookup** — returns return status, refund amount, and return reason for a valid Order ID, correctly distinguishing “no return requested” from an in-progress or completed return.
- **Stock Check** — resolves by exact product ID or partial, case-insensitive product name, and correctly branches into in-stock, low-stock ($\leq$ reorder threshold), and out-of-stock (with restock date) states.

- **FAQ page** — surfaces eight common questions in an expandable format requiring no data lookup.
- **Error handling** — an unmatched Order ID or product query returns a clear “not found” message with a hint, instead of a blank result or a stack trace.

4.2 Known Limitations

Limitation	Detail
Static data source	Data is static CSV, not a live order-management or inventory system — a production version needs a real data integration.
No authentication	Any Order ID can be looked up by anyone who has it, which is acceptable for an MVP demo but not for production; should be paired with the account/email on the order.
No deflection analytics	No analytics yet on which questions are deflected vs. escalated, so deflection rate can't be measured quantitatively from the app itself.
Single-language, text-only	No voice or multilingual support in the current interface.

5. Team Process and Work Distribution

Delivery followed the Team Charter's structure: a 10-task board (each task capped at four hours per the anti-black-box rule), same-day status updates, and a conflict-resolution ladder (direct discussion → pod-level consensus → lead escalation) that was never needed past its first step.

The board balanced two tasks per seat across all five seats, with high-priority items — charter, board setup, repository creation, and core implementation — front-loaded into the first half of the sprint so that downstream tasks (dashboard, report, audit log, go-live note) had a stable foundation to build on.

Note: this report presents the aggregate delivery outcome only; individual contribution breakdowns (owner, task, commit/PR evidence) are tracked separately in the audit log (Task 9).

6. Next Steps

- **Live data integration** — replace static CSVs with a live connection to Northstar's order management and inventory systems.
- **Identity verification** — add a lightweight identity check (email or account match) before returning order or return details.
- **Deflection instrumentation** — log lookup volume and “not found” rate as a measurable proxy for deflection impact.
- **FAQ expansion** — grow FAQ content based on real post-launch question logs rather than the initial assumed set.